MODULE 2

1. What is a Software Process?

A **Software Process** is a structured set of activities used to **develop, deliver, and maintain software**.

It defines **how software is developed**, from understanding requirements to designing, coding, testing, deployment, and maintenance.

Main activities:

- Requirement analysis
- Design
- Implementation/Coding
- Testing
- Deployment
- Maintenance

---

2. What is SDLC? Explain its Phases with a Neat Diagram.

SDLC (Software Development Life Cycle) is a framework that describes the **activities performed at each stage of a software development project**.

Phases of SDLC

1. **Requirement Analysis** – Identifying the information, functions, behavior, performance, and interfaces required by the customer.
2. **Design** – Designing the data structures, software architecture, user interfaces, and algorithms.
3. **Implementation** – Converting the design into source code and developing the required software components.
4. **Testing** – Checking the software to find errors and ensuring that it works according to the requirements.
5. **Deployment** – Installing and delivering the completed software to the users.

6. **Maintenance** – Correcting errors and making necessary enhancements or modifications after deployment.

3. Why is SDLC Important?

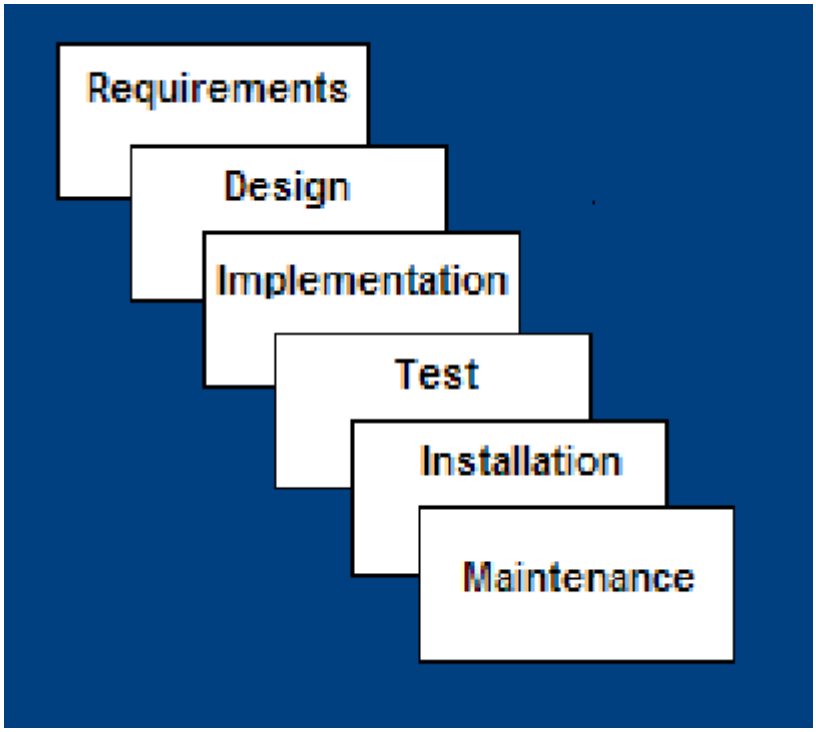
SDLC is important because it provides a systematic and organized approach to software development. It helps the development team plan, develop, test, and maintain software properly.

Importance of SDLC:

1. **Proper Planning**
   SDLC divides software development into different phases, making the project easier to plan and manage.
2. **Better Quality**
   Testing and review activities help identify and fix errors, resulting in better-quality software.
3. **Risk Reduction**
   Potential problems and risks can be identified during the development process and addressed early.
4. **Time and Cost Management**
   Proper planning of activities and resources helps control the project's time and cost.
5. **Clear Responsibilities**
   Each phase has specific activities, making it easier for team members to understand their responsibilities.
6. **Customer Satisfaction**
   SDLC ensures that customer requirements are properly understood and incorporated into the software.
7. **Easy Maintenance**
   A structured development process makes it easier to modify, improve, and maintain the software after delivery.

4. Waterfall Model

The **Waterfall Model** is a **linear and sequential software development model** in which development proceeds through a series of phases. Each phase is completed before moving to the next phase. It is suitable when the **requirements are well known, the product definition is stable, and the technology is understood**.



Phases of the Waterfall Model

1. Requirements

In this phase, all the required **information, functions, behavior, performance, and interfaces** of the software are identified and documented.

2. Design

The software is designed based on the requirements. This includes **data structures, software architecture, interface representations, and algorithmic details**.

3. Implementation

The design is converted into actual software. This phase involves **source code development, database creation, user documentation, and testing activities**.

4. Testing

The developed software is tested to identify defects and verify that the system works according to the specified requirements.

5. Installation

The completed software is installed and delivered to the intended environment or users.

6. Maintenance

After delivery, the software is maintained by making necessary **corrections, modifications, and enhancements**.

Advantages of the Waterfall Model

1. **Easy to understand and use** because the phases are clearly defined and sequential.
2. **Provides structure to inexperienced staff.**
3. **Milestones are clearly understood**, making project progress easier to track.
4. **Requirements remain stable** throughout development.
5. **Good management control** because planning, staffing, and tracking can be done phase by phase.
6. Works well when **quality is more important than cost or schedule**.

Disadvantages of the Waterfall Model

1. **All requirements must be known upfront.**
2. Deliverables from each phase are considered **frozen**, which reduces flexibility.
3. It can give a **false impression of progress**.
4. It does not properly reflect the **problem-solving and iterative nature** of software development.
5. **Integration occurs at the end**, which can make problems difficult to detect early.

6. There is **little opportunity for customers to preview the system** until late in development.

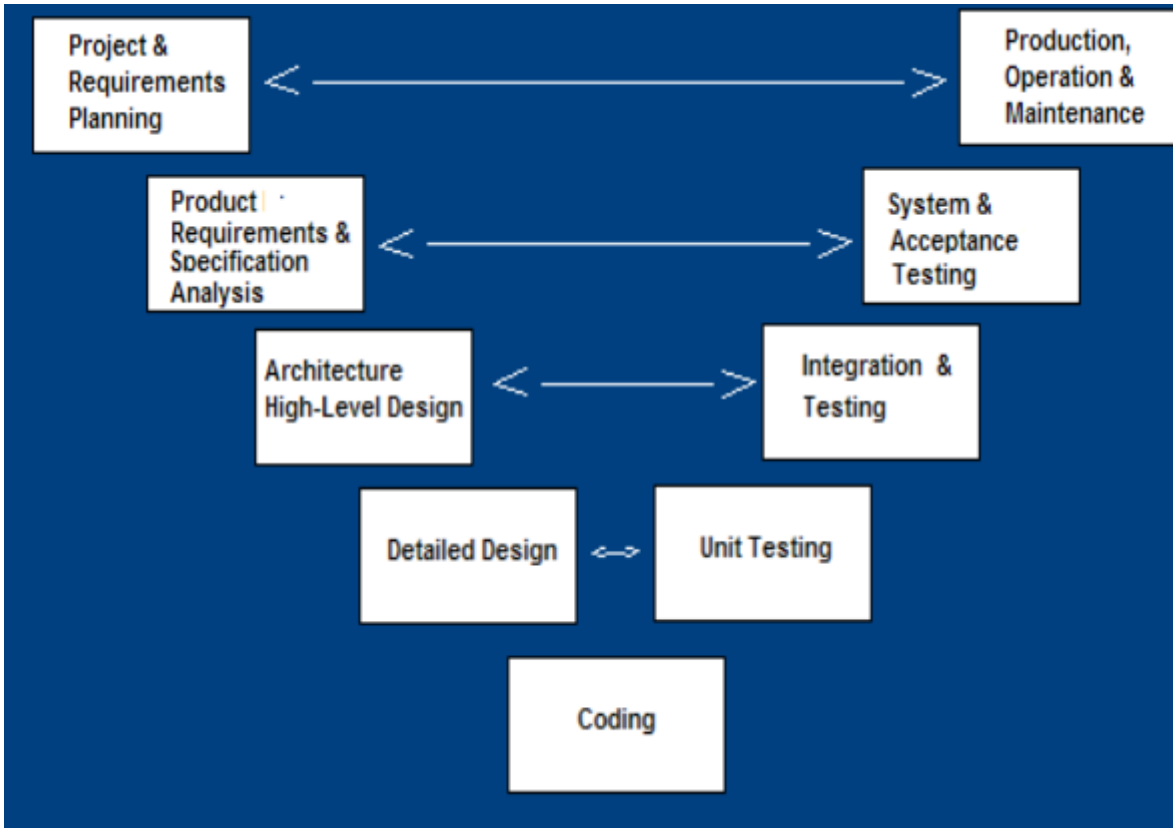
When Should the Waterfall Model Be Used?

The Waterfall Model should be used when:

- **Requirements are very well known.**
- **Product definition is stable.**
- **Technology is well understood.**
- Developing a **new version of an existing product**.
- **Porting an existing product to a new platform**.

5. V-Model

The **V-Model (Verification and Validation Model)** is a variant of the **Waterfall Model** that emphasizes **verification and validation** of the software product. In this model, testing activities are planned in parallel with the corresponding development phases.



Phases of the V-Model

1. Project and Requirements Planning

Resources required for the project are planned and allocated.

2. Product Requirements and Specification Analysis

A complete specification of the software system is prepared.

3. Architecture / High-Level Design

The overall architecture of the software is designed, defining how the software functions will fulfill the requirements.

4. Detailed Design

Detailed algorithms and designs are developed for each architectural component.

5. Coding

The algorithms and designs are converted into actual software code.

6. Unit Testing

Each individual module is tested to check whether it performs as expected.

7. Integration and Testing

The different modules are combined and tested to ensure that they interconnect correctly.

8. System and Acceptance Testing

The complete software system is tested in its intended environment to verify that it satisfies the requirements.

9. Production, Operation and Maintenance

The software is put into operation and maintained through necessary enhancements and corrections.

Advantages of the V-Model

1. **Early planning for verification and validation**
   Testing and verification are considered from the early stages of development.
2. **Each deliverable is testable**
   Every development phase has a corresponding testing activity.
3. **Easy progress tracking**
   Project management can track progress using clearly defined milestones.
4. **Easy to use**
   The model is structured and straightforward to understand.

Disadvantages of the V-Model

1. **Does not easily handle concurrent events.**
2. **Does not handle iterations or phases easily.**
3. **Does not easily handle changing requirements.**
4. **Does not contain risk-analysis activities.**

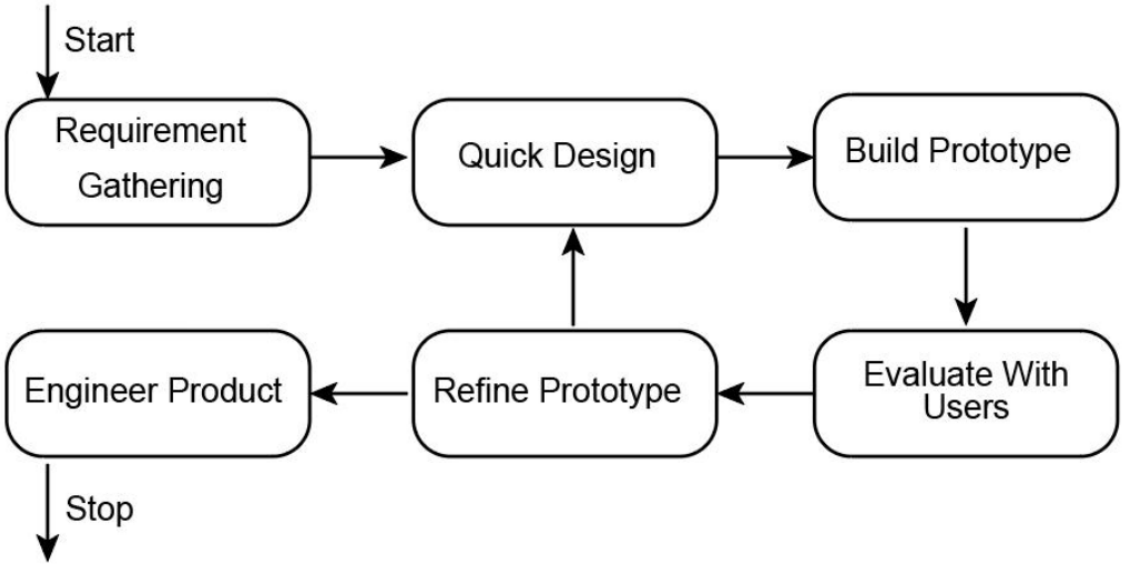
When is the V-Model Preferred?

The V-Model is preferred:

- For **systems requiring high reliability**, such as hospital patient-control applications.
- When **all requirements are known upfront**.
- When changing requirements can be handled beyond the analysis phase.
- When the **solution and technology are already known**.

6. Prototype Model/ Structured Evolutionary Prototyping

The **Prototype Model** is a software development model in which a **working prototype is built during the requirements phase** and shown to the users for evaluation. The users provide feedback, and the prototype is refined repeatedly until the requirements are clearly understood and the user is satisfied.



1. Requirement Gathering

The developer collects the **basic requirements** from the users. The main purpose is to understand what the user expects from the software.

2. Quick Design

A **simple initial design** of the software is prepared. It focuses on the important features and gives an idea of the system's interface and functionality.

3. Build Prototype

A **prototype** is developed using the requirements and quick design. It is an early working version of the proposed software.

4. Evaluate with Users

The prototype is given to the users for evaluation. Users check the prototype and provide **feedback, corrections, and suggestions**.

5. Refine Prototype

The developer improves the prototype according to the feedback received from the users. Changes are made to the design and functionality wherever required.

6. Repeat the Process

If the users are not satisfied, the prototype is refined again and evaluated again. This process continues until the requirements are properly understood and the users are satisfied.

7. Engineer Product

Once the prototype is satisfactory, the developer develops the **actual final software product** using proper engineering practices.

8. Stop

After the final product has been developed, the prototype development cycle is completed.

In simple words:

The Prototype Model works by developing an initial prototype, getting user feedback, improving it repeatedly, and finally developing the actual software product.

Advantages of Prototype Model

1. **Better understanding of requirements** – Customers can see the system while requirements are being gathered.
2. **Customer involvement** – Developers receive direct feedback from customers.
3. **More accurate final product** – Continuous feedback helps produce a system closer to user expectations.
4. **Handles unexpected requirements** – New requirements can be identified and accommodated.
5. **Flexible design and development** – Changes can be made during the refinement process.

6. **Visible progress** – Users can see a working prototype instead of waiting until the complete system is developed.
7. **Improves functionality awareness** – Interaction with the prototype helps users identify additional functionality they need.

Disadvantages of Prototype Model

1. **Code-and-fix development** – Developers may abandon structured development and focus only on quickly modifying the prototype.
2. **Quick-and-dirty development** – The prototype approach may encourage poor development practices.
3. **Maintainability may be overlooked** – The prototype may not be designed with long-term maintenance in mind.
4. **Customer may demand the prototype** – The customer may want the prototype itself to be delivered as the final product.
5. **Process may continue forever** – Repeated changes and feedback can make the development process continue indefinitely.

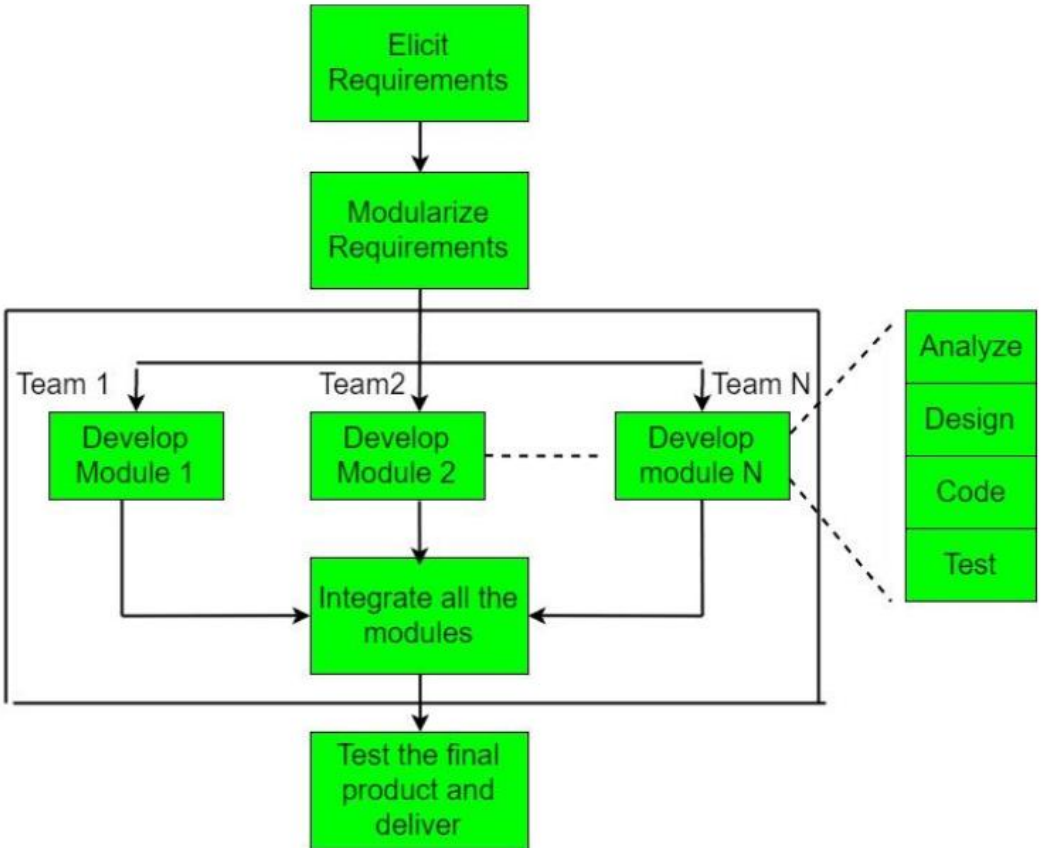
When Should the Prototype Model Be Used?

The Prototype Model should be used when:

- **Requirements are unstable.**
- Requirements need to be **clarified or better understood**.
- A **short-lived demonstration** of the proposed system is required.

7. RAD Model

RAD (Rapid Application Development) is an **incremental software development model** that focuses on a very short development cycle. It is used when requirements are well understood and the software can be developed using a **component-based construction approach**.



Phases of RAD Model

1. Requirements Gathering

The requirements of the software are collected and understood.

2. Analysis and Planning

The requirements are analyzed and the project is planned. The system is divided into smaller modules.

3. Design

Each module is designed according to its requirements.

4. Build / Construction

The modules are developed by separate teams. Each module goes through basic activities such as **analysis, design, coding, and testing**.

5. Integration

All independently developed modules are combined to form the complete software system.

6. Testing and Deployment

The complete product is tested and then delivered to the users.

Important: RAD generally uses a short **time-box of about 60–90 days** for delivery.

Advantages of RAD Model

1. **Reduced development time** – The model focuses on a short development cycle.
2. **Improved productivity** – Different modules can be developed by different teams simultaneously.
3. **Lower cost** – Reduced development time and fewer people can help reduce costs.
4. **Time-box approach** – Helps control project cost and schedule risks.
5. **Customer involvement** – The customer is involved throughout the development cycle.
6. **Modular development** – The project can be divided into small modules and developed independently.
7. **Focus on working software** – RAD gives greater focus to developing the actual software rather than excessive documentation.

Disadvantages of RAD Model

1. **Requires quick user responses** because development happens rapidly.
2. **Risk of never achieving closure** if continuous changes prevent completion.
3. **Difficult to use with legacy systems.**
4. **Requires a modular system** so that the project can be divided into independent components.
5. **Requires strong commitment** from both developers and customers because activities have to be completed within a short time frame.

When Should the RAD Model Be Used?

The RAD Model should be used when:

- **Requirements are reasonably well known.**
- The **user is involved throughout the life cycle.**
- The project can be **time-boxed**.
- Functionality can be **delivered in increments**.
- **High performance is not required.**
- **Technical risks are low.**
- The system can be **modularized**.

8. What is CMM?

CMM (Capability Maturity Model) is a **benchmark used to measure the maturity of an organization's software development process**.

It helps an organization evaluate how well its software processes are defined, managed, controlled, and improved.

CMM defines **five levels of process maturity**, with each level representing an improvement in the organization's software development practices.

---

8. Explain the Five Maturity Levels of CMM

CMM consists of **five maturity levels:**

Level 1 – Initial

At this level, the software development process is **unstructured and unpredictable**.

- There is no well-defined standard process.
- Success mainly depends on individual skills and efforts.
- Approximately **70% of organizations** are shown at this level in the chapter.

Level 2 – Repeatable

At this level, basic project management processes are established so that successful practices can be **repeated for similar projects**.

Important activities include:

- Requirements management
- Software project planning
- Software project tracking and oversight
- Software quality assurance
- Software configuration management

Level 3 – Defined

At this level, software development processes are **properly documented, standardized, and defined across the organization**.

Important activities include:

- Peer reviews
- Intergroup coordination
- Software product engineering
- Integrated software management
- Training programs
- Organization process definition
- Organization process focus

Level 4 – Managed

At this level, the organization **measures and controls its software processes quantitatively**.

Important activities include:

- Software quality management
- Quantitative process management

Level 5 – Optimizing

This is the **highest maturity level**. The organization focuses on **continuous process improvement**.

Important activities include:

- Process change management
- Technology change management
- Defect prevention

9. Advantages / Applications of CMM

Note: Your chapter does **not provide a separate list of advantages/applications of CMM**. It mainly defines CMM and explains its five maturity levels.

Based on the purpose of CMM, you can write these applications:

1. **Measures process maturity** – Helps determine the maturity level of an organization's software processes.
2. **Improves software processes** – Provides a structured path for improving development practices.
3. **Improves software quality** – Better processes help organizations produce more reliable software.
4. **Better project management** – Encourages proper planning, tracking, and management of software projects.
5. **Reduces defects** – Higher maturity levels focus on quality management and defect prevention.
6. **Supports continuous improvement** – Organizations can gradually progress from Level 1 to Level 5.
7. **Standardizes processes** – Helps establish consistent and well-defined software development practices across an organization.

In short:

CMM helps an organization evaluate its current software process maturity and systematically improve its processes from the Initial level to the Optimizing level.

10. What is CMMI?

CMMI (Capability Maturity Model Integration) is the successor to CMM and is a more evolved model. It integrates the best components of different disciplines such as Software CMM, Systems Engineering CMM, and People CMM.

11. Explain the Objectives of CMMI.

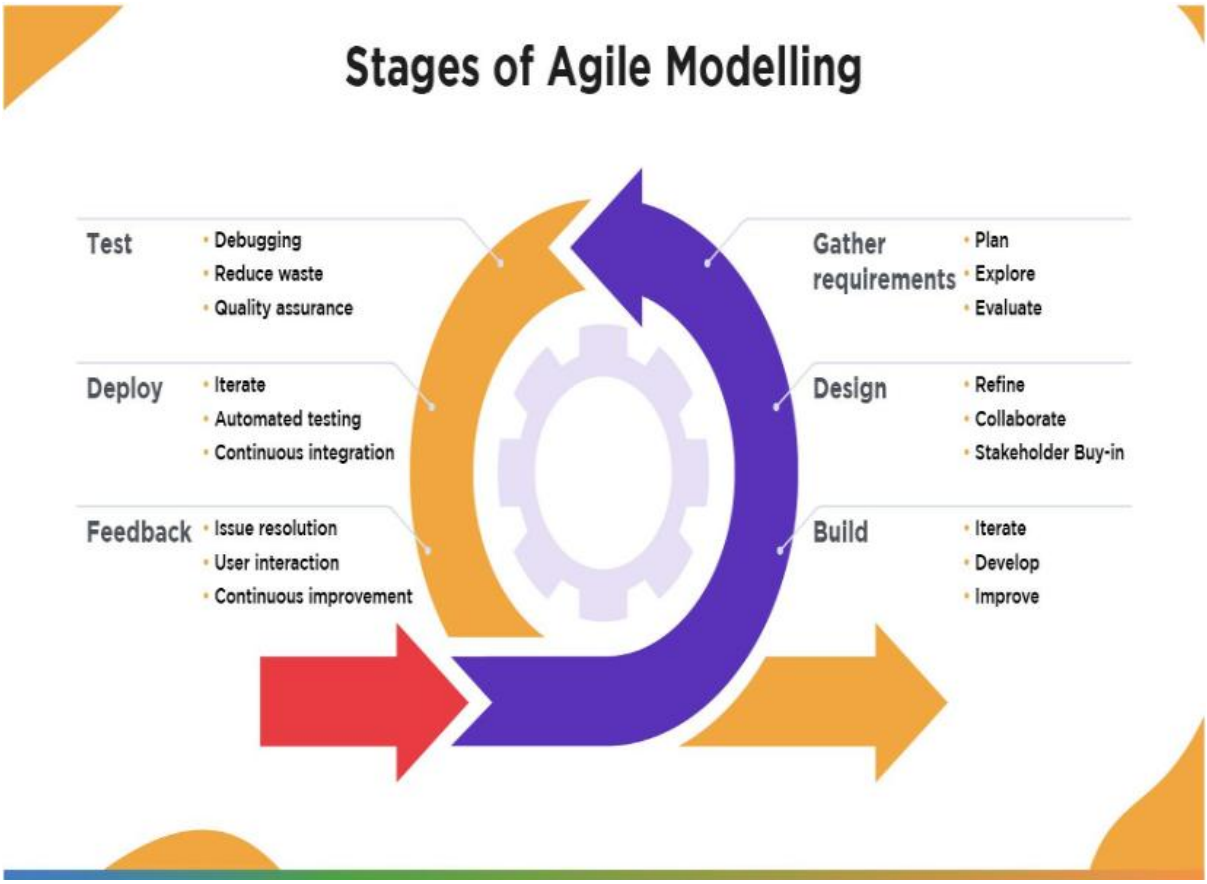
1. Fulfilling customer needs and expectations.
2. Creating value for investors/stockholders.
3. Increasing market growth.
4. Improving the quality of products and services.
5. Enhancing the reputation of the organization in the industry.

12. Differentiate between CMM and CMMI.

CMM	**CMMI**
1. CMM stands for Capability Maturity Model.	CMMI stands for Capability Maturity Model Integration.
2. CMM is an older model.	CMMI is the successor of CMM.
3. CMM focuses on a specific discipline.	CMMI integrates multiple disciplines.
4. CMM makes integration of different disciplines difficult.	CMMI makes integration of different disciplines easier.
5. CMM is a reference model for a specific discipline.	CMMI is a more evolved and integrated model.
6. CMM models are considered separately for different disciplines.	CMMI allows multiple disciplines to be integrated as needed.
7. CMM has a narrower scope.	CMMI has a broader scope.
8. CMM is less integrated.	CMMI provides an integrated approach.

13. Why was Agile introduced? Explain Agile Methodology with a neat diagram.

Agile was introduced to support faster software development, especially for **time-critical applications**, by reducing or bypassing unnecessary life-cycle phases. Agile focuses on flexibility, customer involvement, continuous communication, and frequent delivery of working software.



Stages of Agile Methodology

1. **Gather Requirements**
   Requirements are collected, explored, and evaluated to understand what the customer needs.
2. **Design**
   The requirements are refined and the team collaborates with stakeholders to prepare the design.
3. **Build**
   The software is developed in small iterations. The team continuously develops and improves the product.

4. **Feedback**
   Users provide feedback and issues are resolved. Continuous improvement is performed based on user interaction.
5. **Deploy**
   The developed software is deployed. Iterative development, automated testing, and continuous integration are used.
6. **Test**
   The software is tested through debugging, waste reduction, and quality assurance.

The process is **iterative**, meaning the team continuously uses feedback to improve the software in subsequent cycles.

Main Principles of Agile

- Customer satisfaction is given the highest priority.
- Changing requirements are accepted, even late in development.
- Working software is delivered frequently.
- Business people and developers work together.
- Motivated individuals are given proper support.
- Face-to-face communication is encouraged.
- Working software is the primary measure of progress.
- Continuous attention is given to technical excellence and good design.
- Self-organizing teams help improve architecture and design.
- Simplicity is important in development.

14. Differentiate between Agile and Prescriptive Process Models.

Agile Process Models	Prescriptive Process Models
1. More flexible and adaptable.	1. More structured and predefined.
2. Accept changing requirements.	2. Changes are generally more difficult to accommodate.
3. Focus on frequent delivery of working software.	3. Follow predefined phases and processes.
4. Customer involvement is continuous.	4. Customer involvement is generally more structured.
5. Less formal.	5. More formal.
6. Emphasizes communication and collaboration.	6. Emphasizes defined processes and documentation.
7. Suitable for time-critical applications.	7. Suitable when requirements and processes can be clearly defined.
8. Development is iterative and incremental.	8. Development generally follows a more planned and sequential approach.

15. Explain Adaptive Software Development (ASD), its Steps, Advantages and Limitations.

Adaptive Software Development (ASD) is an Agile approach that combines **Rapid Application Development (RAD)** with software engineering best practices. It develops software through repeated cycles, where the success of one cycle is reviewed before planning the next cycle.

Steps of ASD

1. **Project Initialization** – Determine the purpose and intent of the project.
2. **Determine Project Time-Box** – Estimate the overall duration of the project.
3. **Determine Number of Cycles** – Decide the number of development cycles and the time-box for each cycle.
4. **Write Objective Statement** – Define the objective to be achieved in each cycle.
5. **Assign Components** – Assign the primary software components to each cycle.
6. **Develop Project Task List** – Prepare the list of tasks required for the project.
7. **Review the Cycle** – Review and evaluate the success of the completed cycle.
8. **Plan the Next Cycle** – Use the review results to plan the next development cycle.

Advantages of ASD

1. **Flexible development** – The process can adapt to changes during development.
2. **Iterative development** – Software is developed through repeated cycles.
3. **Continuous improvement** – Each completed cycle is reviewed before planning the next one.
4. **Better project control** – Time-boxes help organize and control development activities.

5. **Early feedback** – Reviewing each cycle helps identify problems and improvements early.

Limitations of ASD

Note: These limitations are not explicitly listed in your uploaded PDF; they are standard limitations of ASD.

1. **Requires experienced team members** – The team must be capable of making decisions and adapting to changes.
2. **Difficult to predict the final product** – Requirements and plans can change during development.
3. **Requires continuous customer involvement** – Regular feedback and interaction may be necessary.
4. **Less suitable for highly regulated projects** – Projects requiring strict, fixed processes and documentation may find ASD difficult to use.
5. **Scope may change frequently** – Continuous adaptation can make it difficult to maintain a fixed scope, schedule, and cost.

16. Explain Extreme Programming (XP), its Practices/Features, Advantages and Disadvantages.

Extreme Programming (XP) is an Agile software development method mainly suitable for **small-to-medium-sized teams** working on software with **vague or rapidly changing requirements**. In XP, **coding is a key activity throughout the software project**, and communication, testing, and system behavior are strongly supported through code.

Practices / Features of XP

1. **Planning Game** – Determines the scope of the next release using business priorities and technical estimates.
2. **Small Releases** – A simple system is released first, followed by new versions in short cycles.
3. **Metaphor** – Development is guided by a simple shared story describing how the system works.

4. **Simple Design** – The system is designed as simply as possible, and unnecessary complexity is removed.
5. **Testing** – Programmers continuously write unit tests, while customers write tests for features.
6. **Refactoring** – The system is continuously restructured without changing its behavior to remove duplication and simplify the code.
7. **Pair Programming** – Production code is written by two programmers working together at one computer.
8. **Collective Ownership** – Any programmer can modify any part of the code when required.
9. **Continuous Integration** – The system is integrated and built multiple times a day whenever tasks are completed.
10. **40-Hour Week** – Developers should normally work no more than 40 hours per week.
11. **On-Site Customer** – A customer or user is available as part of the team to answer questions.
12. **Coding Standards** – Programmers follow common coding rules to make the code easier to understand and maintain.

Advantages of XP

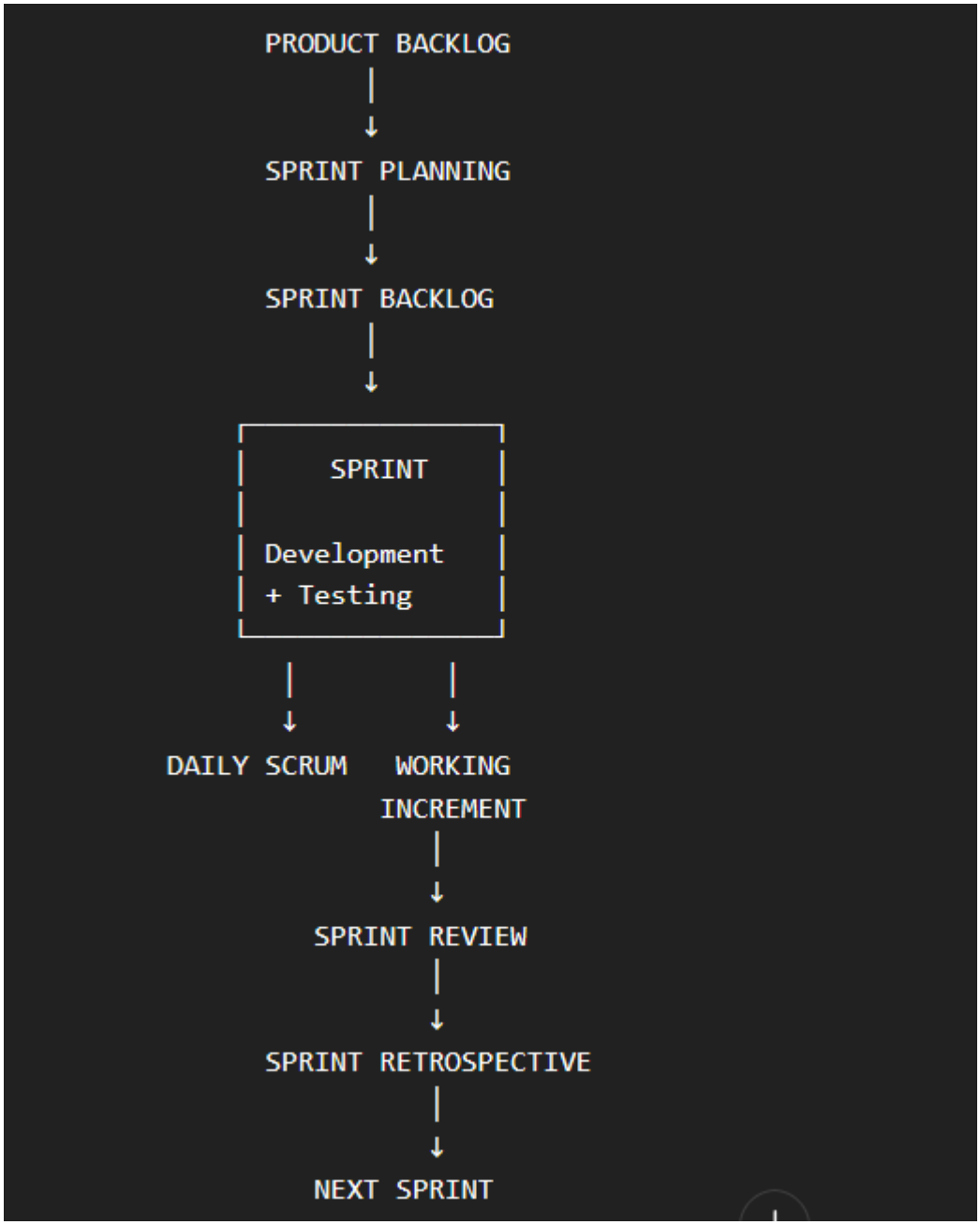
- **Frequent software delivery** through small releases.
- **Better code quality** through continuous testing and refactoring.
- **Early detection of errors** through continuous testing.
- **Improved communication** through pair programming and on-site customer involvement.
- **Easy adaptation to changing requirements** because development occurs in short cycles.
- **Reduced complexity** through simple design and refactoring.

Disadvantages of XP

- Requires **continuous customer involvement**.
- Pair programming can require **more developer resources**.
- Frequent changes can make **long-term planning difficult**.
- Requires a **highly skilled and disciplined team**.
- Continuous testing and integration require significant effort.
- Less suitable for **large teams** or projects where requirements are highly stable.

16. Explain the Scrum Framework with a neat diagram.

Scrum is an Agile framework used to develop software through **short, iterative development cycles called Sprints**. It focuses on continuous development, customer feedback, and delivering a working product increment.



Working of Scrum

1. **Product Backlog** – Contains all required features, improvements, and tasks of the product.
2. **Sprint Planning** – The team selects the work that will be completed during the upcoming Sprint.
3. **Sprint** – The selected work is developed and tested during a fixed period.
4. **Daily Scrum** – The team meets regularly to discuss progress, problems, and the work to be done.
5. **Increment** – At the end of the Sprint, a working part of the product is produced.
6. **Sprint Review** – The completed work is demonstrated and feedback is collected.
7. **Sprint Retrospective** – The team reviews the Sprint and identifies ways to improve the next Sprint.

17. Explain the Scrum Roles

1. Product Owner

The **Product Owner** represents the customer and is responsible for maximizing the value of the product.

Main responsibilities:

- Manages and prioritizes the **Product Backlog**.
- Defines and clarifies product requirements.
- Communicates customer needs to the development team.
- Decides which features are most important.

2. Scrum Master

The **Scrum Master** helps the team follow Scrum practices and removes obstacles that affect the team's progress.

Main responsibilities:

- Facilitates Scrum meetings.
- Helps the team follow Scrum principles.
- Removes obstacles faced by the team.
- Supports the Product Owner and Development Team.

3. Development Team

The **Development Team** consists of professionals who design, develop, test, and deliver the product.

Main responsibilities:

- Develop the selected features.
- Perform testing and integration.
- Work together to complete Sprint goals.
- Produce the working Increment.

---

18. Explain Scrum Events

1. Sprint

A **Sprint** is a fixed period during which the team develops and delivers a usable product Increment.

2. Sprint Planning

The team decides:

- What work will be done during the Sprint.
- How the selected work will be completed.

3. Daily Scrum

A short daily meeting where team members discuss:

- What they completed.
- What they will work on next.
- Any problems or obstacles.

4. Sprint Review

At the end of the Sprint, the team presents the completed Increment to stakeholders and receives feedback.

5. Sprint Retrospective

The team discusses the completed Sprint and identifies:

- What went well.
- What problems occurred.
- What can be improved in the next Sprint.

---

19. Explain Scrum Artifacts

1. Product Backlog

The **Product Backlog** is a prioritized list containing all the features, requirements, improvements, and other work needed for the product.

2. Sprint Backlog

The **Sprint Backlog** contains the items selected from the Product Backlog that the team plans to complete during the current Sprint.

3. Increment

The **Increment** is the completed and working portion of the product produced during a Sprint. It should meet the required quality standards and contribute to the final product.

Note: Your uploaded 69-page chapter only mentions **Scrum as one of the Agile methods** and does not provide these detailed Scrum roles, events, or artifacts. The detailed Scrum questions themselves are present in your Module 2 question list.

20. Explain the Incremental SDLC Model, its advantages, disadvantages, and when it should be used.

Incremental SDLC Model is a software development model in which a **partial implementation of the complete system is developed first**, and additional functionality is gradually added through successive releases. The requirements are prioritized and implemented in groups until the complete system is developed.

Advantages

1. High-risk or major functions can be developed first.
2. Each release provides an **operational product**.
3. Customers can provide feedback on each build.
4. Uses a **divide-and-conquer** approach.
5. Reduces the initial delivery cost.
6. Provides faster initial product delivery.
7. Customers receive important functionality early.
8. Reduces the risk caused by changing requirements.

Disadvantages

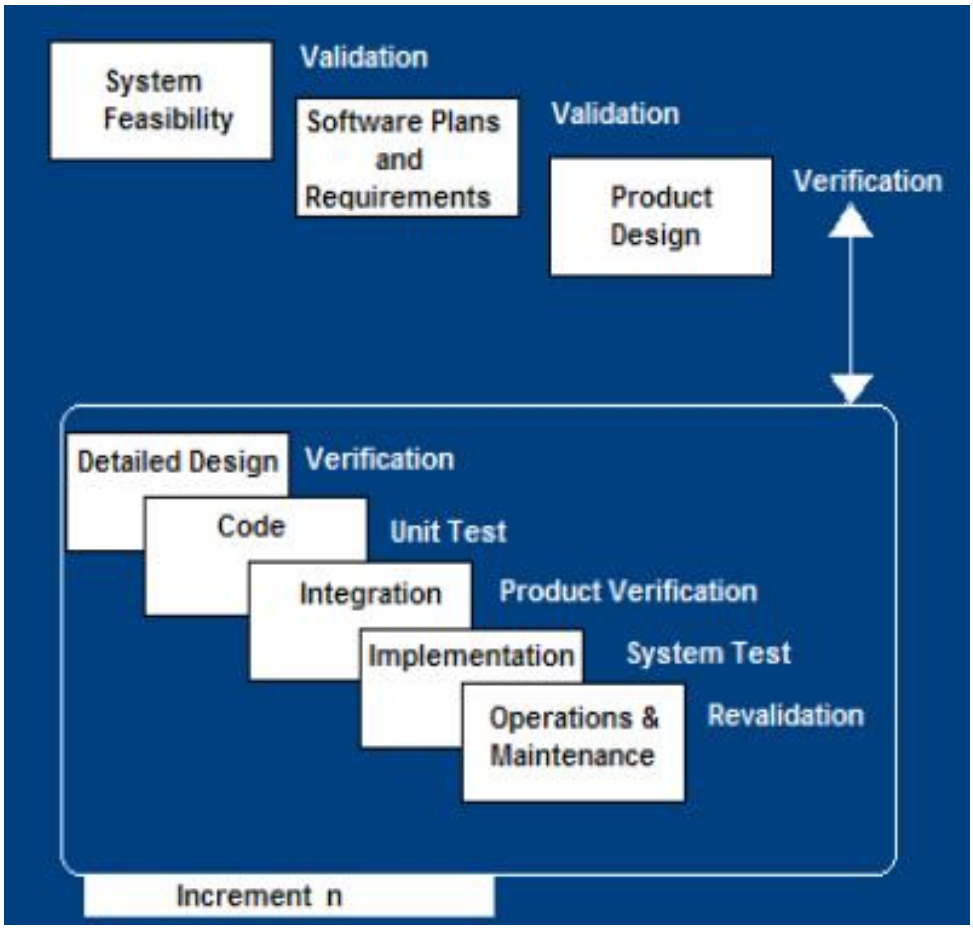
1. Requires **good planning and design**.
2. Requires early definition of the complete system to identify the increments.
3. Requires **well-defined module interfaces**.
4. The total cost of the complete system is **not necessarily lower**.

When should the Incremental Model be used?

It should be used when:

- There are **risk, funding, schedule, or complexity concerns**.
- There is a need for **early realization of benefits**.
- Most requirements are known initially but may **evolve over time**.
- There is a need to get **basic functionality to the market early**.
- The project has a **lengthy development schedule**.
- The project uses **new technology**.

Diagram:-



1. System Feasibility

First, the **feasibility of the complete system** is studied. The organization determines whether the system is practical and possible to develop.

2. Software Plans and Requirements

The software requirements are identified and planned. The requirements are **prioritized and divided into different increments**.

3. Product Design

The overall product design is prepared based on the identified requirements.

4. Detailed Design

For the selected increment, a detailed design is created. The specific components and functionality to be developed are defined.

5. Code

The design of the current increment is converted into **source code**.

6. Unit Test

Each individual module or component is tested to make sure it works correctly.

7. Integration

The developed modules are combined with the existing system and checked to ensure they work together properly.

8. Product Verification

The integrated increment is verified to ensure that it satisfies the specified requirements.

9. Implementation

The completed increment is implemented and added to the existing system.

10. System Test

The system is tested as a complete unit to check whether the increment works correctly with the existing system.

11. Operations & Maintenance

The released increment is operated and maintained. Necessary corrections and improvements are made.

12. Increment n

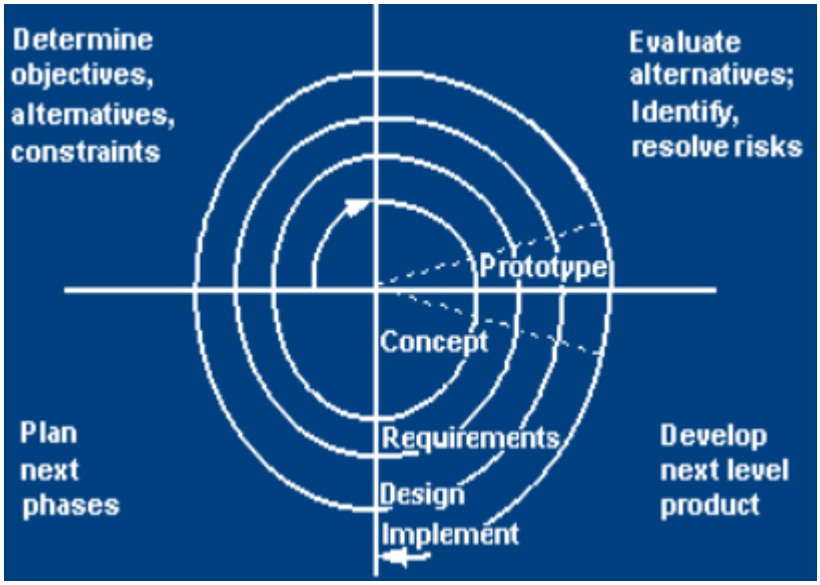
The process is repeated for **Increment 1, Increment 2, ... until Increment n**, where each new increment adds more functionality to the previous release.

What do Validation, Verification and Revalidation mean?

- **Validation:** Checks whether the software being developed meets the intended requirements.
- **Verification:** Checks whether the software is being developed correctly according to its specifications and design.
- **Revalidation:** Checks the system again after a new increment or modification to ensure that existing functionality still works correctly.

21. Explain the Spiral SDLC Model, its four quadrants, advantages, disadvantages, and when it should be used.

The **Spiral Model** is an SDLC model that combines the **Waterfall Model with risk analysis and RAD prototyping**. Each cycle of the spiral follows a sequence of activities similar to the Waterfall process.



Four Quadrants of the Spiral Model

1. **Determine Objectives, Alternatives and Constraints**
   The objectives of the project are identified along with possible alternatives and constraints such as **cost, schedule, and interfaces**.
2. **Evaluate Alternatives and Identify/Resolve Risks**
   Different alternatives are studied and possible risks are identified. These risks may include **lack of experience, new technology, tight schedules, and poor processes**. Appropriate steps are taken to resolve the risks.
3. **Develop the Next-Level Product**
   The next version of the software is developed. Activities include **creating and reviewing the design, developing and inspecting code, and testing the product**.
4. **Plan the Next Phase**
   Plans are prepared for the next cycle, including the **project plan, configuration management plan, test plan, and installation plan**.

Advantages of Spiral Model

1. Provides **early identification of major risks**.
2. Users can see the system early through **rapid prototyping**.
3. **High-risk functions are developed first**.
4. The design does not need to be perfect in the beginning.
5. Users can remain closely involved throughout the development process.

6. Provides **early and frequent user feedback**.
7. Project costs are assessed frequently.

Disadvantages of Spiral Model

1. Risk evaluation can take too much time for **small or low-risk projects**.
2. Planning, risk analysis, and prototyping can become excessive.
3. The model is **complex**.
4. Requires expertise in **risk assessment**.
5. The Spiral process may continue indefinitely.
6. Developers may need to be reassigned during non-development activities.
7. It can be difficult to define clear milestones for moving to the next iteration.

When Should the Spiral Model Be Used?

The Spiral Model should be used:

- When **prototyping is appropriate**.
- When **cost and risk evaluation are important**.
- For **medium- to high-risk projects**.
- When users are **unsure about their requirements**.
- When requirements are **complex**.
- For developing a **new product line**.
- When **significant changes are expected**, such as in research and exploration projects.

22. Explain the Concurrent Development Model, its Advantages and Disadvantages.

The **Concurrent Development Model**, also called the **Concurrent Model**, represents software development activities as **states**. Different activities can progress concurrently, and an activity can move from one state to another when changes or events occur.

Working of the Concurrent Development Model

- Each software engineering activity has different **states**.
- For example, an activity may be **under development, under review, under revision, awaiting changes, baselined, or done**.
- Activities can move between these states depending on project events.
- If the customer changes a requirement, the affected activity can move to the **awaiting changes** state.
- This allows different development activities to progress concurrently rather than waiting for one complete sequential phase.

Advantages

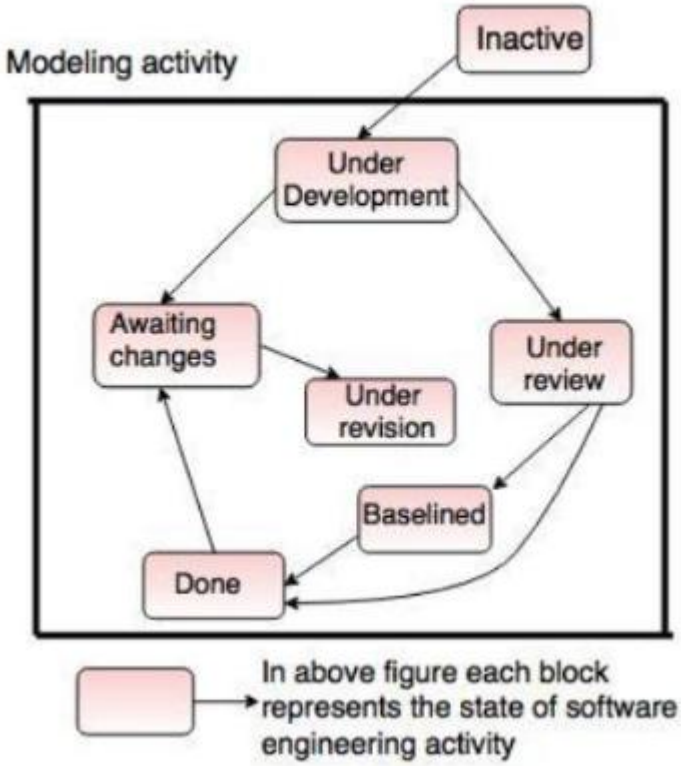
1. **Applicable to different types of software development processes.**
2. **Easy to understand and use.**
3. Provides **immediate feedback from testing**.
4. Provides an **accurate picture of the current state of the project**.

Disadvantages

1. Requires **better communication between team members**, which may not always be achieved.
2. The team needs to **remember and manage the status of different activities**.

Diagram:-

state.



 Inactive
The activity has not started yet or is currently not active.

 Under Development
The activity is currently being developed. For example, the team may be creating the system model.

 Under Review
The developed work is reviewed to check whether it is correct and meets the requirements.

 Under Revision
If problems or changes are identified during the review, the activity is modified or corrected.

 Awaiting Changes
The activity is waiting for some required changes or information before development can continue.

Baselined
The work has been reviewed and approved and becomes a **baseline**, meaning it is treated as an approved version.

Done
The activity has been completed successfully.

23. Explain Feature Driven Development (FDD) and its Five Process Activities.

Feature Driven Development (FDD) is an Agile process model that focuses on developing software through **small, client-valued features**. It follows a structured five-step process for planning, designing, and building features.

Five Process Activities of FDD

1. **Develop an Overall Model**
   Create an overall model of the system to understand its structure and major components.
2. **Build a Features List**
   Identify and list the features that the system needs to provide.
3. **Plan by Feature**
   Plan the development work according to the identified features and assign responsibilities.
4. **Design by Feature**
   Design the individual features before they are implemented.
5. **Build by Feature**
   Develop, test, and integrate the features into the software product.

In short: FDD develops software by **modeling, listing, planning, designing, and building features**.

---

24. Explain Dynamic Systems Development Method (DSDM), including its Principles and Lifecycle.

Dynamic Systems Development Method (DSDM) is an Agile software development approach that focuses on **frequent delivery, active user involvement, and delivering business value within fixed time and cost constraints**.

Principles of DSDM

1. **Focus on business needs.**
2. **Deliver on time.**
3. **Collaborate with stakeholders.**
4. **Never compromise quality.**
5. **Build incrementally from firm foundations.**
6. **Develop iteratively.**
7. **Communicate continuously and clearly.**
8. **Demonstrate control over the development process.**

DSDM Lifecycle

1. **Feasibility Study** – Determine whether the project is technically and economically feasible.
2. **Business Study** – Understand business requirements and processes.
3. **Functional Model Iteration** – Develop and refine the functional model through iterations.
4. **Design and Build Iteration** – Design and build the required system functionality.
5. **Implementation** – Deploy the completed system and put it into operation.

In short: DSDM emphasizes **business value, user involvement, iterative development, fixed time and cost, and continuous delivery**.

---

25. Explain the Tailored SDLC Model.

A **Tailored SDLC Model** is a software life-cycle model that is **customized according to the specific requirements and characteristics of a project**.

There is no single SDLC model that fits every project. Therefore, an organization can select and combine suitable practices from different models according to its needs.

Main Points

1. **Customized approach** – The SDLC is adapted to suit the project.
2. **Based on project requirements** – The selected process depends on factors such as project size, complexity, risk, and requirements.
3. **Combines suitable practices** – Useful practices from different SDLC models can be combined.
4. **Improves suitability** – The process is designed to match the specific project environment.

In short: A Tailored SDLC Model means **modifying the development process to fit the needs of a particular project instead of blindly following one standard model**.

---

26. What is Software Quality? Why is Software Quality Important?

Software quality refers to the degree to which software **satisfies its stated requirements and meets the expectations of its users**. Quality includes both explicitly stated requirements and implied expectations.

Importance of Software Quality

1. **Meets requirements** – Ensures the software performs the functions specified by the customer.
2. **Improves reliability** – Reduces failures and unexpected behavior.
3. **Reduces defects** – Helps identify and prevent errors in the software.
4. **Improves user satisfaction** – Software that works correctly and meets expectations provides a better user experience.

5. **Reduces maintenance problems** – Higher-quality software generally requires fewer corrections after delivery.
6. **Improves performance** – Quality processes help ensure that the software performs as expected.

---

27. Explain the Quality Assurance Plan and its Major Activities.

A **Quality Assurance Plan (QAP)** defines the activities and procedures used to ensure that software development follows the required **quality standards and processes**.

Major Activities

1. **Defect Tracking**
   Identifying, recording, and monitoring defects found during development.
2. **Unit Testing**
   Testing individual software components to verify that they work correctly.
3. **Source-Code Tracing**
   Examining the source code to identify errors and ensure that the code follows the required standards.
4. **Technical Reviews**
   Reviewing software work products to identify defects and improve quality.
5. **Integration Testing**
   Testing combined modules to ensure that they work correctly together.
6. **System Testing**
   Testing the complete system to verify that it satisfies the specified requirements

28. Differentiate between Waterfall Model and V-Model

Waterfall Model	V-Model
1. Follows a **linear and sequential** approach.	1. Follows a **V-shaped sequential** approach.
2. Testing is mainly performed **after development**.	2. Testing activities are planned alongside development phases.
3. Focuses mainly on **development phases**.	3. Gives strong emphasis to **verification and validation**.
4. Testing is not explicitly connected to each development phase.	4. Each development phase has a corresponding testing activity.
5. Less suitable for systems requiring extensive verification.	5. Suitable for **high-reliability systems**.
6. Changes in requirements are difficult to handle.	6. Changes in requirements are also difficult to handle.
7. Simpler to understand and implement.	7. More structured because of its verification and validation activities.
8. Suitable when requirements are **well known and stable**.	8. Suitable when requirements are known and **high reliability is required**.

29. Differentiate between Waterfall Model and Prototype Model

Waterfall Model	Prototype Model
1. Follows a **sequential** development approach.	1. Follows an **iterative** development approach.
2. Requirements should be clearly known at the beginning.	2. Useful when requirements are **unclear or unstable**.
3. Customer feedback generally comes later.	3. Customer feedback is obtained **throughout prototyping**.
4. A complete product is developed according to predefined requirements.	4. An early **prototype** is developed first.
5. Changes are difficult to accommodate.	5. Changes can be incorporated through prototype refinement.
6. Testing generally occurs after implementation.	6. The prototype is repeatedly evaluated by users.
7. Less suitable when requirements are uncertain.	7. Well suited for **clarifying requirements**.
8. Suitable when requirements are **stable and well understood**.	8. Suitable when requirements need **clarification or better understanding**.

30. Differentiate between Waterfall Model and RAD Model

Waterfall Model	RAD Model
1. Follows a **linear and sequential** approach.	1. Uses **rapid and incremental development**.
2. Development usually follows one phase at a time.	2. Different modules can be developed **concurrently**.
3. Development generally takes more time.	3. Focuses on a **short development cycle**.
4. Customer involvement is comparatively limited.	4. Requires **active user involvement**.
5. Requirements should be stable and well defined.	5. Requirements should be **reasonably well known**.
6. Does not focus strongly on reusable components.	6. Uses **component-based construction**.
7. Less suitable when quick delivery is required.	7. Suitable when **quick delivery** is important.
8. Suitable for stable projects with clearly defined requirements.	8. Suitable when functionality can be **delivered in increments**.

31. Differentiate between Waterfall Model and Agile Model

Waterfall Model	Agile Model
1. Follows a **sequential** approach.	1. Follows an **iterative and incremental** approach.
2. Requirements are generally defined at the beginning.	2. Requirements can **change during development**.
3. Customer involvement is comparatively limited.	3. Customer involvement is **continuous**.
4. Working software is generally delivered after major development phases.	4. Working software is delivered **frequently**.
5. Testing is mainly performed after implementation.	5. Testing is performed **continuously** during development.
6. Changes are difficult and costly to accommodate.	6. Changes can be accommodated even **late in development**.
7. More formal and documentation-oriented.	7. Less formal and emphasizes **communication and collaboration**.
8. Suitable when requirements are **stable and well understood**.	8. Suitable when requirements are **changing or evolving**.

32. Differentiate between XP and Scrum

Extreme Programming (XP)	Scrum
1. XP is an **Agile development method**.	1. Scrum is an **Agile framework**.
2. Strongly focuses on **software engineering practices**.	2. Strongly focuses on **project management and team organization**.
3. Includes practices such as **pair programming**.	3. Does not require pair programming.
4. Uses **continuous integration** as an important practice.	4. Scrum focuses on delivering an Increment during each Sprint.
5. Includes **Test-Driven Development/testing practices**.	5. Testing practices are not prescribed specifically by Scrum.
6. Has practices such as **refactoring and coding standards**.	6. Defines roles, events, and artifacts.
7. Has an **on-site customer** practice.	7. Has a **Product Owner** responsible for maximizing product value.
8. XP focuses more on **how software is developed**.	8. Scrum focuses more on **how development work is organized and managed**.